L'objet avec TypeScript

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I - Contexte **3**

II - Interfaces **4**

A. Interfaces 4

B. Exercice : Quiz 8

III - Classes en TypeScript **10**

A. Classes en TypeScript 10

B. Exercice : Quiz 14

IV - Les objets en TypeScript **16**

A. Les objets en TypeScript 16

B. Exercice : Quiz 18

V - Essentiel **20**

VI - Auto-évaluation **21**

A. Exercice 21

B. Test 21

Solutions des exercices **23**

Tous droits réservés à STUDI - Reproduction interdite

I Contexte

Durée : 1 h

Prérequis : les bases en TypeScript

Contexte

Lorsqu'on parle de programmation orientée objet en TypeScript, il est important de comprendre les concepts clés tels que les interfaces, les classes et les objets. Ces notions fondamentales constituent une base solide pour optimiser la manipulation et le traitement des données en TypeScript. Dans ce cours, nous allons aborder ces concepts en détail en fournissant des exemples pour une meilleure compréhension.

Nous allons voir d'abord les interfaces qui sont des structures de données qui définissent la forme des objets en décrivant les types de leurs propriétés et de leurs méthodes. Les interfaces jouent un rôle crucial dans la programmation orientée objet en TypeScript en permettant la réutilisation et la modularité du code.

Ensuite, nous allons voir les classes qui sont des modèles pour la création d'objets, qui regroupent des propriétés et des méthodes communes. Elles constituent un moyen efficace de structurer le code en définissant les caractéristiques communes des objets et en permettant la création de multiples instances. Les classes permettent également l'héritage, où une classe peut hériter des propriétés et des méthodes d'une autre classe.

Et enfin, nous allons voir les objets qui sont des instances de classes avec des valeurs concrètes pour leurs propriétés. Ils sont créés à partir des modèles de classe et peuvent être manipulés en utilisant leurs propriétés et méthodes. Les objets jouent un rôle central en TypeScript en permettant la création de structures de données complexes et la réutilisation du code.

La compréhension de ces concepts est essentielle pour développer des applications en TypeScript. Il est également important de maîtriser les concepts d'attributs, de méthodes, d'implémentation et d'instanciation, pour tirer pleinement parti de la programmation orientée objet en TypeScript.

Tous droits réservés à STUDI - Reproduction interdite

II Interfaces

A. Interfaces

Concept d'interface en TypeScript

En TypeScript, les interfaces sont des structures qui permettent de définir des contrats pour les types de données. Elles décrivent la forme que doit prendre un objet et les types des propriétés qu'il doit contenir. Les interfaces peuvent également décrire des fonctions, des classes et des types génériques.

Voici un exemple simple d'interface avec des propriétés et des méthodes :

```
interface Person {
  name: string;
  age: number;
  sayHello: () => void;
}
```

Cette interface Person décrit les propriétés et méthodes d'un objet qui représente une personne. Elle indique que chaque objet de ce type doit avoir une propriété name de type string, une propriété age de type number et une méthode sayHello() qui ne prend pas de paramètres et ne renvoie rien (void).

Cette interface peut être utilisée pour définir des variables ou des paramètres de fonction qui doivent correspondre à la forme de l'objet décrit par l'interface. Elle peut également être utilisée pour créer des classes qui implémentent cette interface et fournissent une implémentation pour la méthode sayHello().

Définir la structure d'objet en utilisant une interface

Voici un exemple de définition de structure d'objet en utilisant une interface :

```
interface Produit {
  nom: string;
  prix: number;
// En TypeScript on utilise le signe "?" pour dire que la propriété est optionnelle
  description?: string;    // La propriété description est optionnelle
  disponible: boolean;
}

const tshirt: Produit = {
  nom: "T-shirt",
  prix: 19.99,
  disponible: true
};
```

Dans cet exemple, l'interface Produit est utilisée pour définir la structure d'un objet représentant un produit. L'objet tshirt respecte cette structure en incluant les propriétés nom, prix, description (optionnelle) et disponible.

Les alias

Un alias de type en TypeScript permet de créer un nouveau nom pour un type existant. Cela permet de simplifier la lecture et l'écriture de types complexes et répétitifs en leur donnant un nom plus court et plus explicite.

Tous droits réservés à STUDI - Reproduction interdite

Il est possible de créer des alias de types en utilisant le mot-clé type. Ces alias de types peuvent être utilisés pour simplifier la syntaxe des types et les rendre plus lisibles. Voici un exemple :

```
type Age = number;
interface Person {
  name: string;
  age: Age;
  sayHello: () => void;
}
```

Dans cet exemple, nous avons créé un alias de type Age pour le type number. Nous avons ensuite utilisé cet alias pour la propriété age de l'interface Person. Cela rend la syntaxe de l'interface plus concise et plus lisible.

Exemple

Les alias de types peuvent également être utilisés pour définir des types génériques. Voici un exemple :

```
type Pair<T, U> = {
  first: T;
  second: U;
}
let pair: Pair<string, number> = {
  first: "one",
  second: 2
};
```

Dans cet exemple, nous avons créé un alias de type Pair qui prend deux paramètres de type T et U et renvoie un objet avec deux propriétés *first* de type T et *second* de type U. Nous avons ensuite créé un objet pair qui correspond à cet alias de type en spécifiant les types string et number pour les paramètres T et U.

Les alias de types permettent de simplifier la syntaxe des types et les rendre plus lisibles. Ils peuvent être utilisés pour définir des types génériques ou pour créer des interfaces qui référencent des types existants. Les interfaces, quant à elles, permettent de décrire les propriétés et méthodes d'un objet et de s'assurer que les variables et paramètres qui y font référence correspondent à cette description.

Interface VS Alias

Les interfaces et les alias de types sont deux fonctionnalités importantes de TypeScript pour définir des types. Voici quelques-unes de leurs particularités.

Les interfaces peuvent être étendues

Les interfaces peuvent être étendues, tandis que les alias de types ne peuvent pas l'être. Cela signifie qu'il est possible de définir une interface qui hérite des propriétés d'une autre interface en utilisant l'opérateur extends, mais cela n'est pas possible avec un alias de type.

L'opérateur extends

Complément

L'opérateur extends permet de créer une classe enfant (ou sous-classe) qui hérite des propriétés et des méthodes de la classe parent (ou super-classe). Cela permet de réutiliser le code existant et de créer des classes plus spécifiques en ajoutant ou en modifiant des fonctionnalités.

Tous droits réservés à STUDI - Reproduction interdite

```typescript
// Exemple d'interface étendue
interface Animal {
  name: string;
  age: number;
}
interface Dog extends Animal {
  breed: string;
}
const myDog: Dog = { name: "Flipper", age: 3, breed: "Labrador" };
```

Dans cet exemple, l'interface Dog étend l'interface Animal pour ajouter une propriété breed.

Les interfaces peuvent définir des propriétés et des méthodes

Les interfaces peuvent définir des propriétés et des méthodes, tandis que les alias de types ne peuvent définir que des types simples ou des types génériques. Cela signifie qu'il est possible d'ajouter des méthodes à une interface, mais cela n'est pas possible avec un alias de type.

```typescript
interface Person {
  name: string;
  age: number;
//  Définir une méthode
  sayHello: () => void;
}
```

Dans cet exemple, l'interface Person définit une méthode sayHello().

Les interfaces sont utilisées pour décrire la forme d'un objet

Les interfaces sont souvent utilisées pour décrire la forme d'un objet, en définissant les propriétés et les méthodes qu'il doit avoir. Cela permet de s'assurer que les objets respectent une structure cohérente et cohésive dans une application.

```typescript
// Exemple d'interface pour décrire la forme d'un objet :
interface Person {
  name: string;
  age: number;
  sayHello: () => void;
}
// Décrire la forme d'un objet
const person: Person = {
  name: 'John',
  age: 30,
  sayHello: function() {
    console.log(`Hello, my name is ${this.name} and I am ${this.age} years old.`);
  }};
person.sayHello(); // Output: "Hello, my name is John and I am 30 years old."
```

Dans cet exemple, l'interface Person est utilisée pour décrire la forme d'un objet qui doit avoir une propriété name de type string, une propriété age de type number et une méthode sayHello qui ne prend aucun argument et ne renvoie rien.

Les alias de types sont utilisés pour créer des noms plus lisibles

Quant aux alias de types, ils sont souvent utilisés pour créer des noms plus lisibles pour des types complexes ou génériques. Par exemple, au lieu d'utiliser un type générique comme Map<string, number>, on peut créer un alias de type pour, tel que type NumberMap = Map<string, number>. Cela rend le code plus lisible et facile à comprendre, surtout lorsqu'on utilise le même type à plusieurs endroits dans le code.

```typescript
type NumberMap = Map<string, number>;
const myMap: NumberMap = new Map();
myMap.set('one', 1);
myMap.set('two', 2);
console.log(myMap.get('one')); // Output: 1
```


Tous droits réservés à STUDI - Reproduction interdite

Dans cet exemple, un alias de type NumberMap est créé pour le type générique Map<string, number>. Cela permet d'utiliser un nom plus clair pour ce type dans le code. Ensuite, un objet myMap de type NumberMap est créé à partir d'une instance de Map et des éléments y sont ajoutés et récupérés à l'aide de la méthode set et de la méthode get.

Les interfaces peuvent être implémentées par des classes

Les interfaces peuvent être implémentées par des classes, tandis que les alias de types ne peuvent pas l'être. Cela signifie qu'il est possible de définir une classe qui implémente une interface, mais cela n'est pas possible avec un alias de type.

```
interface Animal {
  name: string;
  speak: () => string;
}
class Dog implements Animal {
  name: string;
  constructor(name: string) {
    this.name = name;
  }
  speak() {
    return "Woof!";
  }}
const myDog: Animal = new Dog("Rufus");
console.log(myDog.speak()); // Output: "Woof!"
```

Dans cet exemple, une interface Animal est définie pour décrire la forme d'un objet qui doit avoir une propriété name de type string et une méthode speak qui renvoie une chaîne de caractères. Ensuite, une classe Dog est définie qui implémente l'interface Animal en ayant une propriété name et une méthode speak qui renvoie la chaîne de caractères « *Woof!* ». Enfin, un objet myDog de type Animal est créé à partir d'une instance de Dog et la méthode speak est appelée sur cet objet pour renvoyer la chaîne de caractères « *Woof!* ».

Les interfaces sont plus flexibles et peuvent être implémentées par des classes, tandis que les alias de types sont plus simples et sont souvent utilisés pour définir des noms plus lisibles pour des types complexes. Les interfaces sont également plus adaptées aux bibliothèques ou aux frameworks qui nécessitent des types structurés et des signatures de méthode spécifiques, tandis que les alias de types sont plus adaptés aux types d'union, aux types imbriqués et aux types génériques.

Les interfaces peuvent être utilisées pour décrire des types génériques

Les interfaces peuvent également être utilisées pour décrire des types génériques, qui sont des types qui prennent des paramètres de type. Cela peut être utile pour créer des fonctions et des classes génériques qui peuvent travailler avec différents types de données. Voici un exemple :

```
interface Pair<T, U> {
  first: T;
  second: U;
}
let pair: Pair<string, number> = {
  first: "one",
  second: 2
};
console.log(pair.first); // Output: one
console.log(pair.second); // Output: 2
```

Dans cet exemple, nous avons défini une interface générique Pair qui décrit une paire de valeurs de types différents T et U. Nous avons ensuite créé un objet pair qui correspond à cette interface en spécifiant les types string et number pour les paramètres T et U. Enfin, nous avons accédé aux propriétés first et second de l'objet pair pour afficher leurs valeurs dans la console.

Tous droits réservés à STUDI - Reproduction interdite

B. Exercice : Quiz

[solution n°1 p. 23]

Question 1

Qu'est-ce qu'une interface en TypeScript ?

- Une fonction qui prend en entrée des arguments typés
- Une description de la forme qu'un objet doit avoir, incluant des propriétés et des méthodes
- Une classe qui peut être instanciée pour créer des objets

Question 2

Une interface peut être étendue par une autre interface.

- Vrai
- Faux

Question 3

Il n'est pas possible d'hériter d'un alias de type dans une interface.

- Vrai
- Faux

Question 4

Quelle est la principale différence entre une interface et un alias de type en TypeScript ?

- Une interface est utilisée pour décrire la forme des objets, tandis qu'un alias de type est utilisé pour créer un nouveau type basé sur un type existant
- Une interface décrit la forme des objets, tandis qu'un alias de type nomme un type existant ou une combinaison de types.
- Il n'y a pas de différence significative entre une interface et un alias de type

Question 5

Quelle expression est juste sur les interfaces en TypeScript (hors abstraction) ?

- **A** Les interfaces peuvent être utilisées pour définir des propriétés et des méthodes
- **B** Les interfaces ne peuvent pas être implémentées par des classes

Tous droits réservés à STUDI - Reproduction interdite

C Les interfaces peuvent hériter des propriétés d'autres interfaces

Tous droits réservés à STUDI - Reproduction interdite

III Classes en TypeScript

A. Classes en TypeScript

Les classes en TypeScript sont une façon de créer des objets complexes en utilisant la syntaxe de classe familière de la Programmation Orientée Objet (POO). Les classes sont souvent utilisées pour créer des objets qui ont des propriétés et des méthodes, ce qui les rend très utiles pour la modélisation d'entités du monde réel dans le code.

Implémentation de classes en TypeScript

En TypeScript, les classes peuvent être implémentées de la même manière que dans d'autres langages de programmation orientés objet, tels que Java ou C#. Les classes sont définies à l'aide du mot-clé « *class* », suivi du nom de la classe et de l'accolade ouvrante pour définir le corps de la classe. Voici un exemple de classe simple en TypeScript :

```
class Car {
  make: string;
  model: string;
  year: number;
  constructor(make: string, model: string, year: number) {
    this.make = make;
    this.model = model;
    this.year = year;
  }
  drive(): void {
    console.log(`Driving a ${this.make} ${this.model} from ${this.year}`);
  }}
```

Dans cet exemple, nous avons une classe Car qui a trois propriétés : make, model, et year. La classe a également un constructeur qui prend ces trois propriétés en tant que paramètres et les affecte aux propriétés correspondantes de l'objet. Enfin, la classe a une méthode drive qui affiche un message décrivant la voiture en train de conduire.

Instanciation de classes (création d'objets)

L'instanciation de classes consiste à créer une nouvelle instance ou objet à partir de la classe. La classe est comme un modèle ou un plan pour l'objet, et l'instanciation crée une copie de ce modèle avec ses propres valeurs de propriétés et de méthodes. En TypeScript, l'instanciation se fait à l'aide du mot-clé « *new* » suivi du nom de la classe et de ses paramètres de constructeur si nécessaire :

```
const myCar = new Car('Tesla', 'Model S', 2022);
```

Maintenant, nous avons une instance de la classe Car qui a les propriétés make, model et year définies avec les valeurs passées dans le constructeur.

Nous pouvons ensuite appeler les méthodes de la classe et accéder aux propriétés de la voiture :

```
myCar.drive(); // affiche "Driving a Tesla Model S from 2022"
```

Dans cet exemple, nous créons une instance de la classe Car en passant « *Tesla* », « *Model S* », et « *2022* » au constructeur. Ensuite, nous appelons la méthode drive de l'objet pour afficher un message.

Les constructeurs en TypeScript

En TypeScript, un constructeur est une méthode spéciale d'une classe qui est appelée lorsqu'une nouvelle instance de cette classe est créée. La fonction du constructeur est d'initialiser les propriétés de l'objet nouvellement créé et de définir son état initial.

Tous droits réservés à STUDI - Reproduction interdite

Exemple

Le constructeur est défini dans la classe à l'aide du mot-clé « *constructor* ». Il peut prendre des paramètres, qui sont utilisés pour fournir des valeurs initiales aux propriétés de l'objet. Par exemple :

```
class Person {
  nom: string;
  age: number;

  constructor(nom: string, age: number) {
    this.nom = nom;
    this.age = age;
  }
}
```

Dans cet exemple, la classe Person a un constructeur qui prend deux paramètres, nom et age, qui sont utilisés pour initialiser les propriétés nom et age de l'objet nouvellement créé.

Lorsqu'une instance d'une classe est créée, le constructeur est automatiquement appelé pour initialiser les propriétés de l'objet. Par exemple :

```
let personne = new Person("Jordan", 30);
```

Dans cet exemple, une nouvelle instance de la classe Person est créée avec les valeurs nom et age définies sur « *Jordan* » et 30, respectivement. Le constructeur est appelé automatiquement pour initialiser les propriétés de l'objet.

Les constructeurs peuvent également être hérités par les sous-classes en utilisant le mot-clé « *super* ». Cela permet aux sous-classes d'appeler le constructeur de la classe parent pour initialiser les propriétés héritées en plus de leurs propres propriétés. Par exemple :

```
class Employe extends Person {
  salaire: number;

  constructor(nom: string, age: number, salaire: number) {
    super(nom, age);
    this.salaire = salaire;
  }
}
```

Dans cet exemple, la classe Employe hérite de la classe Person et a également une propriété salaire. Le constructeur de Employe prend les mêmes paramètres que celui de Person, mais utilise également super(nom, age) pour appeler le constructeur de Person et initialiser les propriétés héritées.

Les méthodes statiques dans une classe

Les classes en TypeScript peuvent également avoir des méthodes statiques, qui sont des méthodes qui appartiennent à la classe plutôt qu'à une instance spécifique de la classe. Voici un exemple :

```
class MathUtils {
  static add(a: number, b: number): number {
    return a + b;
  }}
const sum = MathUtils.add(2, 3);
console.log (sum); // 5
```

Dans cet exemple, nous avons une classe MathUtils qui a une méthode statique add. Cette méthode prend deux paramètres numériques et retourne leur somme. Nous appelons ensuite la méthode statique directement sur la classe elle-même, plutôt que sur une instance de la classe.

Tous droits réservés à STUDI - Reproduction interdite

L'héritage en TypeScript

L'héritage est un concept clé de la programmation orientée objet, qui permet de créer une classe à partir d'une autre classe existante. En TypeScript, on peut utiliser le mot-clé « *extends* » pour définir une classe qui hérite d'une autre classe. Rappelons que l'opérateur extends permet de créer des classes enfant qui hérite des propriétés et des méthodes de la classe parent, afin d'éviter la duplication de code (boilerplate code).

Voici un exemple de classe parente Animal avec une propriété name et une méthode makeSound() :

```
class Animal {
  name: string;
  constructor(name: string) {
    this.name = name;
  }
  makeSound() {
    console.log("The animal makes a sound");
  }}
```

Ensuite, nous pouvons créer une classe enfant Dog qui étend la classe parente Animal et ajoute une méthode bark :

```
class Dog extends Animal {
  bark() {
    console.log("The dog barks");
  }}
```

Maintenant, nous pouvons créer une instance de Dog et accéder à ses propriétés et méthodes. Dans cet exemple, la classe Dog hérite de la propriété name et de la méthode makeSound() de la classe Animal. La classe Dog ajoute ensuite sa propre méthode bark() pour représenter une fonctionnalité spécifique à un chien.

```
// Crée une instance de la classe Dog
const myDog = new Dog("Flipper");
console.log(myDog.name); // Output: Flipper
myDog.makeSound(); // Output: The animal makes a sound
myDog.bark(); // Output: The dog barks
```

Lorsque l'on crée une instance de la classe Dog avec new Dog('Flipper'), la propriété name est initialisée avec la valeur « *Flipper* » grâce au constructeur de la classe Animal. On peut ensuite appeler la méthode bark() de Dog et la méthode makeSound() héritée de Animal.

Dans cet exemple, la classe Dog hérite de la classe Animal. Cela signifie que Dog a accès aux propriétés et aux méthodes définies dans Animal, comme la propriété name et la méthode makeSound(). La classe Dog définit également sa propre méthode bark(), qui n'est pas présente dans Animal.

Implémentation d'interface en TypeScript

L'implémentation d'interface en TypeScript est un moyen de définir un contrat entre une classe et une interface, afin de garantir que la classe implémente toutes les propriétés et méthodes requises par l'interface.

Voici un exemple :

```
interface Animal {
  name: string;
  numberOfLegs: number;
  makeSound: () => void;
}
class Cat implements Animal {
  name: string;
  numberOfLegs: number;
  constructor(name: string) {
    this.name = name;
```


Tous droits réservés à STUDI - Reproduction interdite

```
    this.numberOfLegs = 4;
  }
  makeSound() {
    console.log("Meow");
  }}
```

Dans cet exemple, nous avons une interface Animal qui définit les propriétés et méthodes que toute classe Animal doit implémenter. Ensuite, nous avons une classe Cat qui implémente l'interface Animal. La classe Cat doit donc avoir une propriété name de type string, une propriété numberOfLegs de type number, et une méthode makeSound qui ne prend pas d'arguments et ne retourne rien.

En utilisant une interface pour définir le contrat entre la classe et l'interface, nous pouvons garantir que la classe implémente toutes les propriétés et méthodes requises par l'interface, ce qui rend le code plus sûr et plus facile à maintenir.

Implémentation de plusieurs interfaces

⚠ Attention

En TypeScript, il est possible de définir une classe qui implémente plusieurs interfaces. Cela permet de définir les propriétés et méthodes de la classe en utilisant les contrats définis par les interfaces, ce qui offre une grande flexibilité et facilite la réutilisation du code.

L'avantage de l'implémentation de classe avec des interfaces est qu'elle offre une grande flexibilité et facilite la réutilisation du code. Si d'autres classes doivent également implémenter les mêmes interfaces, elles peuvent simplement les implémenter et utiliser les mêmes propriétés et méthodes définies dans les interfaces, sans avoir besoin de réécrire le code.

Implémentation d'interface VS Héritage en TypeScript

⚠ Attention

L'héritage permet à une classe enfant de prendre en charge les propriétés et les méthodes d'une classe parente, tandis que l'implémentation d'une interface dans une classe implique que la classe implémente toutes les propriétés et méthodes définies dans l'interface.

L'héritage est une relation « *est un* » (par exemple, un chat est un animal), tandis que l'implémentation d'interface est une relation « *peut faire* » (par exemple, une classe « *robot* » peut implémenter l'interface « *déplacer* »).

En termes de syntaxe, l'héritage utilise le mot-clé « *extends* » pour définir la relation parent-enfant, tandis que l'implémentation d'interface utilise le mot-clé « *implements* » pour indiquer que la classe implémente une interface donnée.

Si nous voulons créer une nouvelle classe qui est basée sur une classe existante, l'héritage est probablement le meilleur choix. Cela nous permet de réutiliser le code existant et de le modifier pour répondre à vos besoins spécifiques. Si nous voulons simplement décrire la forme d'un objet sans nous soucier de son implémentation, une interface est probablement le meilleur choix. Cela nous permet de spécifier les propriétés et les méthodes que nous attendons d'un objet sans avoir à nous soucier de la façon dont cet objet est implémenté.

Aussi, il est important de noter qu'il est possible d'hériter que d'une seule classe, mais nous pouvons implémenter plusieurs interfaces, ce qui permet de rendre le code plus réutilisable et plus facile à maintenir.

Tous droits réservés à STUDI - Reproduction interdite

B. Exercice : Quiz

[solution n°2 p. 24]

Question 1

Qu'est-ce qu'une classe en TypeScript ?

- Une fonction spéciale qui renvoie un objet
- Une interface pour définir la structure d'un objet
- Une collection de propriétés et de méthodes qui peuvent être utilisées pour créer des objets

Question 2

Comment hérite-t-on d'une classe en TypeScript ?

- Avec le mot-clé « *implement* »
- Avec le mot-clé « *class* » et le mot-clé « *extends* »
- Avec le mot-clé « *type* » et le mot-clé « *extends* »

Question 3

Comment implémenter une interface par une classe en TypeScript ?

- Avec le mot-clé « *interface* »
- Avec le mot-clé « *type* » et le mot-clé « *implements* »
- Avec le mot-clé « *class* » et le mot-clé « *implements* »

Question 4

Comment peut-on définir une propriété dans une classe ?

- Avec un nom de propriété uniquement
- Avec un type et un nom de propriété
- Avec un type uniquement

Question 5

Quelle expression est juste sur les classes en TypeScript ?

Tous droits réservés à STUDI - Reproduction interdite

A ☐ Les classes ne peuvent pas être utilisées pour créer des objets avec des propriétés et des méthodes

B Les classes peuvent être héritées par d'autres classes à l'aide du mot-clé « *extends* »

C Les classes peuvent être instanciées pour créer des objets spécifiques

Tous droits réservés à STUDI - Reproduction interdite

IV Les objets en TypeScript

A. Les objets en TypeScript

En TypeScript, les objets sont des structures de données complexes qui contiennent des propriétés et des méthodes. Les propriétés sont des variables qui stockent des données, tandis que les méthodes sont des fonctions qui effectuent des actions sur ces données.

Déclaration d'objet en TypeScript

En TypeScript, on peut déclarer un objet en utilisant une notation littérale d'objet, qui consiste à énumérer les propriétés et leurs valeurs entre accolades, comme dans l'exemple suivant :

```
// Déclaration d'un objet
let person = {
  name: "John",
  age: 30,
  greeting: function() {
    console.log("Hello, my name is " + this.name + " and I am " + this.age + " years old.");
  }};
// Utilisation de l'objet
person.greeting(); // Output: Hello, my name is John and I am 30 years old.
```

Dans cet exemple, nous avons créé un objet person qui contient trois propriétés : name, age et greeting. La propriété greeting est une méthode qui affiche un message de salutation en utilisant les valeurs des propriétés name et age.

Définir la structure d'objet en utilisant une interface

En TypeScript, les interfaces sont utilisées pour définir la structure d'un objet et les types de ses propriétés. Cela permet de s'assurer que les objets que nous créons ou que nous manipulons ont les propriétés requises, avec les types appropriés.

Nous pouvons définir la structure de l'objet en utilisant une interface. Pour utiliser une interface pour définir la structure d'un objet, nous devons définir une interface qui décrit les propriétés de l'objet, ainsi que leurs types.

Supposons que nous ayons un code TypeScript qui définit une interface Person avec des propriétés name et age de type string et number, respectivement, et une méthode greeting qui ne prend pas de paramètres et ne renvoie rien (void) :

```
// Déclaration d'une interface
interface Person {
  name: string;
  age: number;
  greeting: () => void;
}
```

Nous pouvons utiliser cette interface pour définir un objet person qui correspond à la structure Person. Nous pouvons le faire en déclarant une variable person de type Person et en initialisant ses propriétés avec des valeurs de type string et number, respectivement, et en définissant la méthode greeting avec une fonction anonyme qui utilise console.log pour afficher un message de salutation :

```
// Déclaration d'un objet avec l'interface Person
let person: Person = {
  name: "John",
  age: 30,
  greeting: function() {
    console.log("Hello, my name is " + this.name + " and I am " + this.age + " years old.");
  }};
```


Tous droits réservés à STUDI - Reproduction interdite

Maintenant, si nous voulons appeler la méthode greeting de l'objet person, nous pouvons simplement utiliser la notation « . » (point) pour accéder à la méthode.

```
// Utilisation de la méthode
person.greeting(); // Output: Hello, my name is John and I am 30 years old.
```

Lorsque nous appelons cette méthode, nous verrons le message de salutation dans la console, qui indique « *Hello, my name is John and I am 30 years old.* ». Dans cet exemple, nous avons déclaré une interface Person qui décrit la structure de l'objet. Ensuite, nous avons créé un objet person en utilisant cette interface.

En utilisant une interface pour décrire les types de paramètres de fonctions ou de méthodes, nous pouvons nous assurer que les paramètres passés ont les propriétés requises, avec les types appropriés.

Création d'objets à partir d'une classe

En TypeScript, les classes sont utilisées pour créer des objets qui partagent une structure et des comportements communs. Pour créer un objet à partir d'une classe, nous devons instancier la classe en utilisant le mot-clé « *new* ».

Création de la classe Person :

```
class Person {
  name: string;
  age: number;
  constructor(name: string, age: number) {
    this.name = name;
    this.age = age;
  }
  sayHello() {
    console.log(`Hello, my name is ${this.name} and I am ${this.age} years old.`);
  }}
```

Dans cet exemple, nous avons créé une classe Person qui contient deux propriétés (name et age). La classe a également un constructeur qui prend deux paramètres (name et age), et qui initialise les propriétés de la classe avec ces valeurs. La classe a également une méthode sayHello() qui affiche un message de salutation en utilisant les propriétés name et age.

Exemple

Pour créer un objet à partir de cette classe, nous devrons instancier la classe en utilisant le mot-clé new. Voici un exemple :

```
let person = new Person("John", 30);
person.sayHello(); // Output: Hello, my name is John and I am 30 years old.
```

Dans cet exemple, nous avons créé un objet person à partir de la classe Person en utilisant le mot-clé « *new* ». Nous avons passé les valeurs « *John* » et 30 au constructeur de la classe pour initialiser les propriétés de l'objet. Ensuite, nous avons appelé la méthode sayHello() de l'objet pour afficher un message de salutation en utilisant les propriétés de l'objet.

Tous droits réservés à STUDI - Reproduction interdite

Objets imbriqués en TypeScript

En TypeScript, il est possible de définir des objets imbriqués, c'est-à-dire des objets qui ont des propriétés qui sont elles-mêmes des objets. Voici un exemple pour illustrer cela.

Supposons que nous ayons une interface Address qui définit les propriétés d'une adresse :

```
interface Address {
  street: string;
  city: string;
  postalCode: string;
}
```

Nous pouvons ensuite utiliser cette interface comme type de propriété pour une interface Person qui définit les propriétés d'une personne, y compris une propriété address qui est un objet de type Address :

```
interface Person {
  name: string;
  age: number;
  address: Address;
}
```

Nous pouvons alors créer un objet person qui correspond à cette structure en spécifiant les propriétés name, age et address, comme suit :

```
// Déclaration d'un objet imbriqué
let person: Person = {
  name: "John",
  age: 30,
  address: {
    street: "123 Main St",
    city: "Anytown",
    postalCode: "12345"
  }};
```

Nous pouvons accéder aux propriétés de l'objet person à l'aide de la notation « . » (point), comme dans l'exemple suivant :

```
// Utilisation de l'objet imbriqué
console.log(person.name); // Output: John
console.log(person.address.city); // Output: Anytown
```

Dans cet exemple, nous avons ajouté une propriété address à l'objet person. La propriété address contient un autre objet avec des propriétés de type adresse. Nous pouvons accéder aux propriétés de l'objet imbriqué en utilisant un point.

Dans cet exemple, person.address.city accède à la propriété city de l'objet address qui est une propriété de l'objet person.

B. Exercice : Quiz

[solution n°3 p. 26]

Question 1

Qu'est-ce qu'un objet en TypeScript ?

- Une collection de propriétés
- Un constructeur de classes
- Un type de données primitif

Tous droits réservés à STUDI - Reproduction interdite

Question 2

Comment créer un objet à partir d'une classe en TypeScript ?

- En appelant la méthode create() de la classe
- En appelant la méthode build() de la classe
- En appelant le constructeur de la classe avec le mot-clé new

Question 3

Quelle est la méthode pour utiliser une interface pour définir un objet en TypeScript ?

- En définissant un objet qui implémente l'interface
- En utilisant le mot-clé « *new* » avec l'interface
- Il n'est pas possible de créer un objet à partir d'une interface directement

Question 4

Comment déclare-t-on un objet en TypeScript ?

- let obj: Object = {};
- let obj = {};
- let obj: any = {};

Question 5

Il est possible de créer un objet directement à partir d'une interface en TypeScript.

- Vrai
- Faux

Tous droits réservés à STUDI - Reproduction interdite

V Essentiel

Les interfaces en TypeScript permettent de définir des contrats pour les types de données, ce qui peut aider à améliorer la robustesse et la maintenabilité du code.

Les interfaces sont plus flexibles que les alias, elles peuvent définir des propriétés et des méthodes, être étendues et être implémentées par des classes. Tandis que les alias de types sont plus simples et sont souvent utilisés pour définir des noms plus lisibles pour des types complexes.

Les classes sont un moyen de définir des objets avec des propriétés et des méthodes. Elles peuvent être utilisées pour créer des instances d'objets et pour définir des relations entre des objets. Elles peuvent avoir un constructeur pour initialiser les propriétés de l'objet, ainsi que des méthodes pour effectuer des actions sur l'objet. Les classes peuvent également avoir des méthodes statiques qui appartiennent à la classe plutôt qu'à une instance de la classe.

L'implémentation d'une classe est l'action de créer la classe en écrivant son code et en définissant ses propriétés et méthodes. Cela peut inclure l'héritage d'une classe parente, l'implémentation d'interfaces, la définition de constructeurs et la définition de méthodes spécifiques à la classe.

L'instanciation d'une classe est l'action de créer un objet à partir d'une classe existante. Une classe est un plan pour créer des objets, tandis qu'une instance de classe est l'objet créé à partir de ce plan. Chaque instance de classe a son propre ensemble de propriétés et de méthodes qui lui sont propres et peut être manipulée indépendamment des autres instances de classe créées à partir de la même classe.

Le constructeur est une méthode importante dans une classe en TypeScript, car elle permet de configurer et d'initialiser les propriétés de l'objet nouvellement créé.

L'héritage en TypeScript permet de réutiliser du code et de créer des classes plus spécialisées à partir de classes existantes.

L'implémentation d'interface et l'héritage sont des concepts qui permettent de structurer le code et de réutiliser du code existant. L'héritage est utile lorsque nous voulons créer une nouvelle classe qui est une version modifiée d'une classe existante, tandis que l'implémentation d'interface est utile lorsque nous voulons décrire la forme d'un objet sans se préoccuper de son implémentation.

Tous droits réservés à STUDI - Reproduction interdite

VI Auto-évaluation

A. Exercice

Question 1

[solution n°4 p. 28]

- Créez une interface « *Personne* » avec les propriétés suivantes : nom (string), age (number), adresse (string), email (string).
- Créez un alias de type « *Utilisateur* » qui sera un objet avec les mêmes propriétés que l'interface Personne.
- Créez un objet « *client* » de type Personne avec les valeurs suivantes : { nom: "*Alex*", age: 35, adresse: "*123 rue de la Victoire*", email: "*alex@gmail.com*" }.
- Créez un objet « *utilisateur* » de type Utilisateur avec les mêmes valeurs que « *client* ».
- Modifiez la propriété « *email* » de l'objet « *utilisateur* » pour qu'elle soit égale à « alex123@gmail.com ».
- Affichez les propriétés des deux objets dans la console.

Question 2

[solution n°5 p. 28]

Vous êtes chargé de développer une application pour une entreprise de location de voitures. L'entreprise possède une flotte de différents types de voitures (compactes, berlines, SUV, etc.) et souhaite avoir une application pour gérer la réservation et la location de ces voitures.

1. Créez une classe Voiture avec les propriétés suivantes : marque, modèle, année, prixJournalier, estDisponible. La classe doit également avoir une méthode estDisponible() qui retourne la disponibilité de la voiture.
2. Créez des sous-classes pour différents types de voitures, par exemple : VoitureCompacte, VoitureBerline, VoitureSUV, etc. Chaque sous-classe doit étendre la classe Voiture et définir ses propres propriétés et méthodes, par exemple nombreDePortes pour VoitureCompacte ou capacitéDeRemorquage pour VoitureSUV.
3. Créez une classe Client avec les propriétés suivantes : nom, prenom, adresse, numTel. La classe doit également avoir une méthode afficherDetails() qui affiche les détails du client.
4. Créez quelques objets de test pour chaque classe et utilisez-les pour tester.

B. Test

Exercice 1 : Quiz

[solution n°6 p. 30]

Question 1

Quelle est la différence entre une classe et une interface en TypeScript ?

A

Une classe définit des objets avec des propriétés et des méthodes, tandis qu'une interface définit la structure des objets

B

Une classe peut être instanciée pour créer des objets, tandis qu'une interface ne peut pas être instanciée directement

Tous droits réservés à STUDI - Reproduction interdite

C

Une classe ne peut pas hériter d'autres classes ou interfaces, tandis qu'une interface peut uniquement hériter d'autres interfaces

Question 2

Comment peut-on implémenter une interface dans une classe ?

En déclarant la classe comme implémentant l'interface

En définissant les propriétés et méthodes de l'interface dans la classe

En créant une nouvelle classe qui étend l'interface

Question 3

Quelle est la différence entre une interface et un alias de type en TypeScript ?

A

Une interface peut définir des propriétés et des méthodes, tandis qu'un alias de type ne peut définir que des types simples ou des types génériques

B

Une interface peut être implémentée par des classes, tandis qu'un alias de type ne peut pas l'être

C Une interface peut être étendue, tandis qu'un alias de type ne peut pas l'être

Question 4

Les interfaces sont souvent utilisées pour décrire la forme d'un objet.

Vrai

Faux

Question 5

On peut créer des alias de type à partir d'une interface.

Vrai

Faux

Tous droits réservés à STUDI - Reproduction interdite

Solutions des exercices

Solution n°1

[exercice p. 8]

Question 1

Qu'est-ce qu'une interface en TypeScript ?

Une fonction qui prend en entrée des arguments typés

✓ Une description de la forme qu'un objet doit avoir, incluant des propriétés et des méthodes

Une classe qui peut être instanciée pour créer des objets

Une interface en TypeScript permet de décrire la forme qu'un objet doit avoir, en définissant ses propriétés et ses méthodes.

Question 2

Une interface peut être étendue par une autre interface.

✓ Vrai

Faux

Il est possible d'étendre une interface en TypeScript en utilisant le mot-clé « *extends* ».

Question 3

Il n'est pas possible d'hériter d'un alias de type dans une interface.

Vrai

✓ Faux

Il est possible d'hériter d'un alias de type dans une interface en utilisant le mot-clé « *type* » pour référencer l'alias de type existant. Cela peut être utile pour ajouter des propriétés ou des méthodes à un type existant, tout en conservant ses caractéristiques d'origine.

Question 4

Quelle est la principale différence entre une interface et un alias de type en TypeScript ?

Tous droits réservés à STUDI - Reproduction interdite

✓ Une interface est utilisée pour décrire la forme des objets, tandis qu'un alias de type est utilisé pour créer un nouveau type basé sur un type existant

Une interface décrit la forme des objets, tandis qu'un alias de type nomme un type existant ou une combinaison de types.

Il n'y a pas de différence significative entre une interface et un alias de type

Les interfaces sont utilisées pour décrire la forme des objets, en spécifiant les propriétés et les types de données qu'ils contiennent. Les alias de type, quant à eux, permettent de créer de nouveaux types basés sur des types existants, en leur donnant un nouveau nom et/ou en leur appliquant des contraintes supplémentaires.

Question 5

Quelle expression est juste sur les interfaces en TypeScript (hors abstraction) ?

A Les interfaces peuvent être utilisées pour définir des propriétés et des méthodes

B Les interfaces ne peuvent pas être implémentées par des classes

C Les interfaces peuvent hériter des propriétés d'autres interfaces

Les interfaces en TypeScript sont des contrats qui définissent les formes et les structures des types de données. Elles peuvent être utilisées pour définir des propriétés et des méthodes, peuvent être implémentées par des classes, et peuvent hériter des propriétés d'autres interfaces.

Solution n°2

[exercice p. 14]

Question 1

Qu'est-ce qu'une classe en TypeScript ?

Une fonction spéciale qui renvoie un objet

Une interface pour définir la structure d'un objet

✓ Une collection de propriétés et de méthodes qui peuvent être utilisées pour créer des objets

En TypeScript, une classe est un concept fondamental qui permet de créer des objets avec des propriétés et des méthodes. Les classes sont des modèles pour créer des objets qui ont des caractéristiques et des comportements spécifiques.

Tous droits réservés à STUDI - Reproduction interdite

Question 2

Comment hérite-t-on d'une classe en TypeScript ?

Avec le mot-clé « *implement* »

✓ Avec le mot-clé « *class* » et le mot-clé « *extends* »

Avec le mot-clé « *type* » et le mot-clé « *extends* »

En TypeScript, on peut hériter d'une classe existante en créant une nouvelle classe qui reprend les propriétés et les méthodes de la classe parente. Pour ce faire, on utilise le mot-clé « *extends* » dans la déclaration de la nouvelle classe.

Question 3

Comment implémenter une interface par une classe en TypeScript ?

Avec le mot-clé « *interface* »

Avec le mot-clé « *type* » et le mot-clé « *implements* »

✓ Avec le mot-clé « *class* » et le mot-clé « *implements* »

En TypeScript, on peut implémenter une interface en créant une classe qui définit les propriétés et les méthodes de l'interface. Pour ce faire, on utilise le mot-clé « *implements* » dans la déclaration de la classe.

Question 4

Comment peut-on définir une propriété dans une classe ?

Avec un nom de propriété uniquement

✓ Avec un type et un nom de propriété

Avec un type uniquement

Pour définir une propriété dans une classe en TypeScript, il faut spécifier le type et le nom de la propriété.

Question 5

Quelle expression est juste sur les classes en TypeScript ?

Tous droits réservés à STUDI - Reproduction interdite

A
Les classes ne peuvent pas être utilisées pour créer des objets avec des propriétés et des méthodes

B Les classes peuvent être héritées par d'autres classes à l'aide du mot-clé « *extends* »

C Les classes peuvent être instanciées pour créer des objets spécifiques

TypeScript permet de créer des objets avec des propriétés et des méthodes, une classe peut être héritée par d'autres classes en utilisant le mot-clé « *extends* » et les classes peuvent être instanciées pour créer des objets spécifiques.

Solution n°3

[exercice p. 18]

Question 1

Qu'est-ce qu'un objet en TypeScript ?

✓ Une collection de propriétés

Un constructeur de classes

Un type de données primitif

Un objet en TypeScript est une collection de propriétés, qui peuvent être des valeurs de différents types, y compris d'autres objets, des tableaux, des fonctions, etc. Il peut également contenir des méthodes, qui sont des fonctions associées à l'objet.

Question 2

Comment créer un objet à partir d'une classe en TypeScript ?

En appelant la méthode create() de la classe

En appelant la méthode build() de la classe

✓ En appelant le constructeur de la classe avec le mot-clé new

En TypeScript, on crée un objet à partir d'une classe en utilisant le mot-clé new suivi du nom de la classe et des paramètres du constructeur entre parenthèses "*let nomObjet = new nomClasse(paramètre1, paramètre2, paramètreN) ;*".

Tous droits réservés à STUDI - Reproduction interdite

Question 3

Quelle est la méthode pour utiliser une interface pour définir un objet en TypeScript ?

✓ En définissant un objet qui implémente l'interface

En utilisant le mot-clé « *new* » avec l'interface

Il n'est pas possible de créer un objet à partir d'une interface directement

Les interfaces en TypeScript sont uniquement utilisées pour définir la structure d'un objet et ne peuvent pas être instanciées directement. Pour créer un objet qui suit une interface, il est nécessaire de définir un objet qui implémente cette interface en définissant toutes les propriétés et méthodes requises par l'interface.

Question 4

Comment déclare-t-on un objet en TypeScript ?

let obj: Object = {};

✓ let obj = {};

let obj: any = {};

En TypeScript, on peut déclarer un objet en utilisant une notation littérale d'objet, qui consiste à énumérer les propriétés et leurs valeurs entre accolades.

Question 5

Il est possible de créer un objet directement à partir d'une interface en TypeScript.

Vrai

✓ Faux

Il n'est pas possible de créer un objet directement à partir d'une interface en TypeScript, car une interface définit que la structure de l'objet, mais ne fournit pas d'implémentation. Elle est utilisée pour décrire la forme d'un objet et peut être implémentée par une classe ou un objet.

Tous droits réservés à STUDI - Reproduction interdite

[exercice p. 21] Solution n°4

Ce cas pratique vous permettra de vous familiariser avec la création d'interfaces, d'alias de type et d'objets en TypeScript.

```
// Interface Personne
interface Personne {
  nom: string;
  age: number;
  adresse: string;
  email: string;
}
// Alias de type Utilisateur
type Utilisateur = {
  nom: string;
  age: number;
  adresse: string;
  email: string;
}
// Objet client
let client: Personne = {
nom: "Alex",
age: 35,
adresse: "123 rue de la Victoire",
email: "alex@gmail.com"
};
// Objet utilisateur
let utilisateur: Utilisateur = {
nom: "Alex",
age: 35,
adresse: "123 rue de la Victoire",
email: "alex@gmail.com"
};
// Modification de l'email de l'utilisateur
utilisateur.email = "alex123@gmail.com";
// Affichage des propriétés des deux objets
console.log("Client :", client); // Client : { nom: 'Alex', age: 35, adresse: '123 rue de la Victoire', email: 'alex@gmail.com'}
console.log("Utilisateur :", utilisateur); // Client : { nom: 'Alex', age: 35, adresse: '123 rue de la Victoire', email: 'alex123@gmail.com'}
```

[exercice p. 21] Solution n°5

Ce cas pratique vous permettra de vous familiariser avec la création de classe, l'héritage et l'instanciation d'objet.

```
class Voiture {
      marque: string;
      modele: string;
      annee: number;
      prixJournalier: number;
      disponible: boolean;
      constructor(marque: string, modele: string, annee: number, prixJournalier: number, disponible: boolean) {
        this.marque = marque;
        this.modele = modele;
        this.annee = annee;
        this.prixJournalier = prixJournalier;
        this.disponible = disponible;
      }
      estDisponible(): boolean {
        return this.disponible;
```


Tous droits réservés à STUDI - Reproduction interdite

```
    }
  }
  class VoitureCompacte extends Voiture {
    nombreDePortes: number;
    constructor(marque: string, modele: string, annee: number, prixJournalier: number, disponible: boolean, nombreDePortes: number) {
      super(marque, modele, annee, prixJournalier, disponible);
      this.nombreDePortes = nombreDePortes;
    }
  }
  class VoitureBerline extends Voiture {
    nombreDePlaces: number;

    constructor(marque: string, modele: string, annee: number, prixJournalier: number, disponible: boolean, nombreDePlaces: number) {
      super(marque, modele, annee, prixJournalier, disponible);
      this.nombreDePlaces = nombreDePlaces;
    }
  }
  class VoitureSUV extends Voiture {
    capaciteDeRemorquage: number;
    constructor(marque: string, modele: string, annee: number, prixJournalier: number, disponible: boolean, capaciteDeRemorquage: number) {
      super(marque, modele, annee, prixJournalier, disponible);
      this.capaciteDeRemorquage = capaciteDeRemorquage;
    } }
  class Client {
    nom: string;
    prenom: string;
    adresse: string;
    numTel: string;
    constructor(nom: string, prenom: string, adresse: string, numTel: string) {
      this.nom = nom;
      this.prenom = prenom;
      this.adresse = adresse;
      this.numTel = numTel;
    }
    afficherDetails(): void {
      console.log(`Nom: ${this.nom}, Prenom: ${this.prenom}, Adresse: ${this.adresse}, Numero de telephone: ${this.numTel}`);
    }}
  // Exemple d'utilisation des classes
  const voiture1 = new VoitureCompacte("Renault", "Clio", 2020, 50, true, 5);
  const voiture2 = new VoitureBerline("Peugeot", "308", 2019, 70, false, 4);
  const voiture3 = new VoitureSUV("Ford", "Explorer", 2021, 100, true, 2000);
  const client1 = new Client("Dupont", "Jean", "15 rue des Lilas, Paris", "01 23 45 67 89");
  const client2 = new Client("Martin", "Sophie", "10 avenue des Champs Elysées, Paris", "06 12 34 56 78");
  console.log(voiture1.marque); //Renault
  console.log(voiture1.estDisponible()); // true
  console.log(voiture2.marque); // Peugeot
  console.log(voiture2.estDisponible()); //false
  console.log(voiture3.capaciteDeRemorquage); // 2000
  client1.afficherDetails(); //Nom: Dupont, Prenom: Jean, Adresse: 15 rue des Lilas, Paris, Numero de telephone: 01 23 45 67 89
```

Tous droits réservés à STUDI - Reproduction interdite

Solution n°6

[exercice p. 21]

Question 1

Quelle est la différence entre une classe et une interface en TypeScript ?

A ☐ Une classe définit des objets avec des propriétés et des méthodes, tandis qu'une interface définit la structure des objets

B ☐ Une classe peut être instanciée pour créer des objets, tandis qu'une interface ne peut pas être instanciée directement

C Une classe ne peut pas hériter d'autres classes ou interfaces, tandis qu'une interface peut uniquement hériter d'autres interfaces

Une classe est utilisée pour créer des objets avec des propriétés et des méthodes, qui peuvent être instanciés pour créer des instances de la classe. D'autre part, une interface est utilisée pour définir la structure d'un objet sans implémenter de code. Les classes peuvent hériter d'autres classes ou interfaces, tandis que les interfaces peuvent hériter uniquement d'autres interfaces.

Question 2

Comment peut-on implémenter une interface dans une classe ?

✓ En déclarant la classe comme implémentant l'interface

En définissant les propriétés et méthodes de l'interface dans la classe

En créant une nouvelle classe qui étend l'interface

Pour implémenter une interface dans une classe, on utilise le mot-clé « *implements* » suivi du nom de l'interface. Ensuite, on doit définir toutes les propriétés et méthodes de l'interface dans la classe. Cette méthode permet de garantir que la classe respecte le contrat défini par l'interface.

Question 3

Quelle est la différence entre une interface et un alias de type en TypeScript ?

A ☐ Une interface peut définir des propriétés et des méthodes, tandis qu'un alias de type ne peut définir que des types simples ou des types génériques

Tous droits réservés à STUDI - Reproduction interdite

B ☐ Une interface peut être implémentée par des classes, tandis qu'un alias de type ne peut pas l'être

C Une interface peut être étendue, tandis qu'un alias de type ne peut pas l'être

Les interfaces peuvent définir des propriétés et des méthodes, être implémentées par des classes, et être étendues. Tandis que les alias de type ne peuvent définir que des types simples ou des types génériques, ne peuvent pas être implémentés par des classes, et ne peuvent pas être étendus.

Question 4

Les interfaces sont souvent utilisées pour décrire la forme d'un objet.

✓ Vrai

Faux

Les interfaces sont utilisées pour décrire la forme d'un objet, c'est-à-dire les propriétés qu'il doit avoir et leur type. Elles sont également utilisées pour décrire la forme d'une fonction, c'est-à-dire les paramètres qu'elle doit avoir et leur type, ainsi que la valeur de retour et son type. Les alias de types, en revanche, sont utilisés pour créer des noms plus lisibles pour des types complexes. Par exemple, on peut créer un alias de type « *Age* » pour le type « *number* », pour rendre le code plus lisible.

Question 5

On peut créer des alias de type à partir d'une interface.

Vrai

✓ Faux

On ne peut pas créer des alias de type à partir d'une interface. Les alias de type et les interfaces sont deux concepts distincts en TypeScript.

Tous droits réservés à STUDI - Reproduction interdite